

Spezifikation Universal Control Room Interface (UCRI2)

Version 2.0.0

Inhaltsverzeichnis

1. [Einleitung](#)
2. [Systemarchitektur](#)
 - 2.1. [Architekturstil Messaging](#)
 - 2.2. [Vermittlungsebene](#)
 - 2.3. [Anwendungsebene](#)
 - 2.4. [Sicherheitskonzept](#)
 - 2.5. [Versionierung](#)
 - 2.5.1. [Transportschicht-Versionierung](#)
 - 2.5.2. [App-Versionierung](#)
3. [Kommunikationsprotokoll](#)
 - 3.1. [Übermittlung von Nachrichten](#)
 - 3.2. [Validierung von Nachrichten](#)
 - 3.3. [KT-Register](#)
 - 3.4. [Datenintegrität](#)
 - 3.4.1. [Signierung](#)
 - 3.4.2. [Verschlüsselung](#)
4. [API](#)
 - 4.1. [Client-API](#)
 - 4.1.1. [/token - HTTP GET](#)
 - 4.1.2. [/info - HTTP GET](#)
 - 4.1.3. [/registry - HTTP GET](#)
 - 4.1.4. [/registry/{id} - HTTP GET](#)
 - 4.1.5. [/messaging/send - HTTP POST](#)
 - 4.1.6. [/messaging/receive - HTTP POST](#)
 - 4.1.7. [/messaging/commit - HTTP POST](#)
 - 4.2. [Peer-to-Peer-API \(P2P-API\)](#)
 - 4.2.1. [/token - HTTP GET](#)
 - 4.2.2. [/info - HTTP GET](#)
 - 4.2.3. [/registry - HTTP GET](#)
 - 4.2.4. [/messaging/send - HTTP POST](#)
 - 4.3. [Fehlerbehandlung](#)
 - 4.3.1. [Fehlercodes für Antworten mit HTTP-Statuscode 400](#)
 - 4.3.2. [Fehlercodes für Antworten mit HTTP-Statuscode 401](#)
 - 4.3.3. [Fehlercodes für Antworten mit HTTP-Statuscode 500](#)
5. [Applikationen](#)

1. Einleitung

UCRI steht für Universal Control Room Interface - ein Protokoll für die Kommunikation zwischen zwei oder mehreren Einsatzleitsystemen.

Nach der erfolgreichen Einführung und eingehenden Erprobung in der Praxis wurden Erfahrungen gesammelt und neue Anforderungen identifiziert. Diese machten eine Weiterentwicklung des UCRI-Protokolls auf einer neuen Architekturbasis erforderlich. UCRI2 ist eine vollständig überarbeitete Version, die alle Anwendungsfälle der Vorgängerversion (UCRI 1.1) unterstützt und die Grundlage für zukünftige, flexible Erweiterungen bildet.

Die Ziele für die Entwicklung von UCRI2 sind:

1. Einfache, schnell zu implementierende API
 - 1.1. standardisierte maschinenlesbare Spezifikation
 - 1.2. sichere Zustellung und Verarbeitung von Meldungen
2. Trennung technischer und fachlicher Aspekte
 - 2.1. Einfache Erweiterbarkeit
 - 2.2. Technische Komponenten müssen bei fachlichen Erweiterungen nicht angepasst werden
 - 2.3. Fachliche Erweiterungen können ohne Anpassungen der Infrastruktur erfolgen
3. Messaging/Routing basierte Architektur
 - 3.1. Einfaches Zusammenspiel mehrerer Hersteller
 - 3.2. Unterstützung zentraler als dezentraler Anbindungen
4. Durchgängige Standardisierung
 - 4.1. BSI-konformes Netzarchitektur und -design
 - 4.2. JSON Schema Spezifikation für Nachrichtenvalidierung
 - 4.3. OAuth 2.0 für Authentifizierung und Autorisierung
 - 4.4. RFC 3447 – Kryptographische Verfahren
 - 4.5. RFC 8785 – JSON Canonicalization Scheme (JCS)
 - 4.6. RFC 7517 – JSON Web Key
 - 4.7. ...

Die vorliegende Spezifikation ist in folgende Kapiteln gegliedert:

Systemarchitektur

In diesem Kapitel wird UCRI2 als eine n:m-Kommunikationsarchitektur zwischen verschiedenen Kommunikationsteilnehmern unter Verwendung des Messaging-Architekturstils beschrieben. Die Trennung zwischen der fachlichen Anwendungsebene und der technischen Vermittlungsebene sowie ein umfassendes Sicherheitskonzept wird eingeführt.

Kommunikationsprotokoll

Das UCRI2-Kommunikationsprotokoll definiert zwei Schnittstellen: die Client-API für die Kommunikation zwischen eines Leitstellensystems und der UCRI2-Infrastruktur sowie die P2P-API für die Kommunikation zwischen verteilten Komponenten der UCRI2-Infrastruktur basierend auf Peer-to-Peer-Topologie. In diesem Kapitel werden Mechanismen für Nachrichtenübermittlung und -validierung sowie zur Gewährleistung der Datenintegrität durch kryptographische Verfahren ausführlich beschrieben.

API

Die UCRI2-APIs werden als REST-Services entworfen und in OpenAPI 3.1.0-Spezifikationen dokumentiert. In diesem Kapitel werden Gliederung der APIs in Funktionsbereiche, sowie Semantik der einzelnen API-Endpunkte beschrieben.

Applikationen

Die Beschreibung der UCRI2-Applikationen wird in Form von JSON-Schemata samt begleitender Dokumentation separat geliefert.

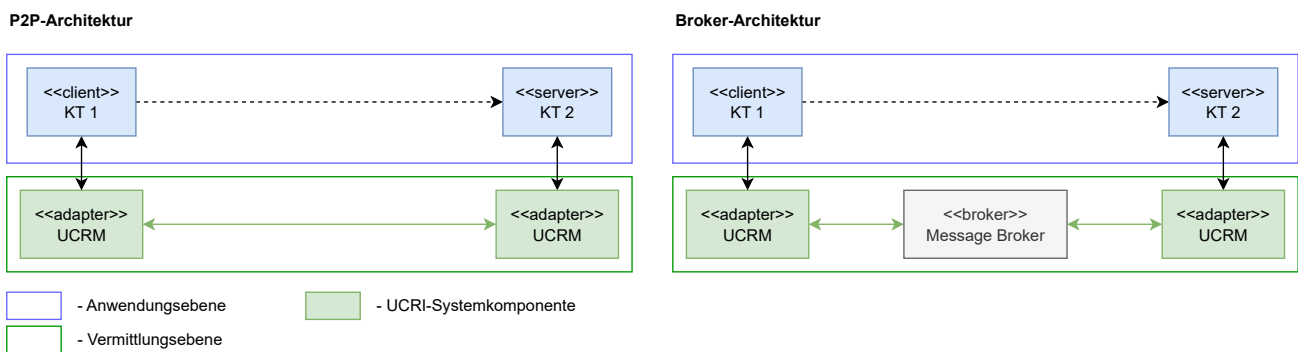
2. Systemarchitektur

Im Gegensatz zu UCRI Version 1, die 1:1-Kommunikation zwischen Leitstellen spezifiziert, ist UCRI 2 grundlegend für die n:m-Kommunikation verschiedener Teilnehmer entwickelt worden.

2.1. Architekturstil Messaging

Bei der Strukturierung der UCRI2-Schnittstelle wird der Architekturstil Messaging verwendet. Bei diesem Architekturstil kommunizieren verteilte unabhängige Systemkomponenten (im allgemeinen Kommunikationsteilnehmer genannt - KT) miteinander mit Hilfe von Nachrichten.

Messaging-Systeme trennen die fachliche Anwendung (gewöhnlich strukturiert nach Client-Server-Prinzip) von der Vermittlungsebene - also von technischen Aspekten der Nachrichtenübermittlung zwischen technischen Systemen der KT. Nachrichten können während der Übertragung umgewandelt werden, ohne dass Sender oder Empfänger von der Umwandlung wissen. Die Entkopplung ermöglicht es Integratoren, je nach Anforderung unterschiedliche Kommunikationstopologien zu implementieren, von dezentralen P2P-Protokollen bis zu zentralisierten Broker-Architekturen:



Messaging-Systeme ermöglichen es den Komponenten, entkoppelt zu bleiben und sich auf ihre eigenen Aufgaben zu konzentrieren, während sie gleichzeitig in der Lage sind, mit anderen Komponenten im System zu kommunizieren und zusammenzuarbeiten. Es verringert die Anzahl der Abhängigkeiten zwischen den Komponenten, wodurch das System flexibler und leichter zu warten ist.

2.2. Vermittlungsebene

Das andere wichtige Architekturmuster, das bei der Strukturierung der UCRI2-Schnittstelle Verwendung findet, ist das Adapter-Muster. Bei diesem Muster erfolgt die Kommunikation zwischen dem technischen System der KT (Anwendungsebene) und der Vermittlungsebene mittels einer Adapter-Komponente Leitstellenmodul (UCRI Control Room Module - UCRM). Der Adapter ermöglicht bidirektionale Kommunikation zwischen den Ebenen in einer standardisierten Form und ermöglicht die Komplexitätsreduzierung der angebundenen Schnittstellen.

Das UCRM stellt die UCRM Client-API bereit - die einzige Kommunikationsschnittstelle für direkt verbundene Kommunikationsteilnehmer wie Leitstellensysteme oder andere technische Knoten, sowie weitere externe Systeme (vgl. [UCRI Gateway](#)).

Die untereinander direkt oder über einen Broker kommunizierenden UCRM-Adapter bilden die Vermittlungsebene und kümmern sich somit um die technischen Aspekte der Kommunikation. Während sich die KT auf die eigentliche Anwendungslogik fokussieren - also die Anwendungsebene.

Zentrale Aufgabe der Vermittlungsebene ist die Zustellung von Meldungen zwischen Sender und Empfänger. Außerdem werden auf der Vermittlungsebene unterschiedliche querschnittliche Aufgaben übernommen.

Einzelne Aufgaben der Vermittlungsebene sind:

1. Verwaltung der Kommunikationstopologie inkl. Adressierungskonzept, KT-Status-Monitoring und KT-Register
2. Konzept Authentisierung, Autorisierung, Accounting
3. Übermittlung von Nachrichten unter der Verwendung des UCRI-Adressierungskonzepts inkl. Routing-Funktion
4. Validierung von Anwendungsmeldungen

Die Vermittlungsebene ist frei von Fachlichkeit. Sie realisiert nur den Datentransport und sichert die Integrität der Daten.

2.3. Anwendungsebene

Die UCRI2-Anwendungsebene ist grundsätzlich getrennt von der Vermittlungsebene. Die Anwendungsebene ist dabei in mehrere unabhängige Applikationen, sogenannte UCRI2-Apps, aufgeteilt.

Eine UCRI2-Applikation wird durch folgende Artefakte definiert:

1. Ein Satz von standardisierten Nachrichten-Schemata (JSON, kanonisches Datenmodell)
2. Ablaufmodell (definierte Abfolge von Nachrichten) ~~###TODO~~
3. Prozessdefinitionen (Festlegungen bezüglich Anwendungslogik, die bei der Implementierung in technischen KT-Systemen berücksichtigt werden müssen) ~~###~~

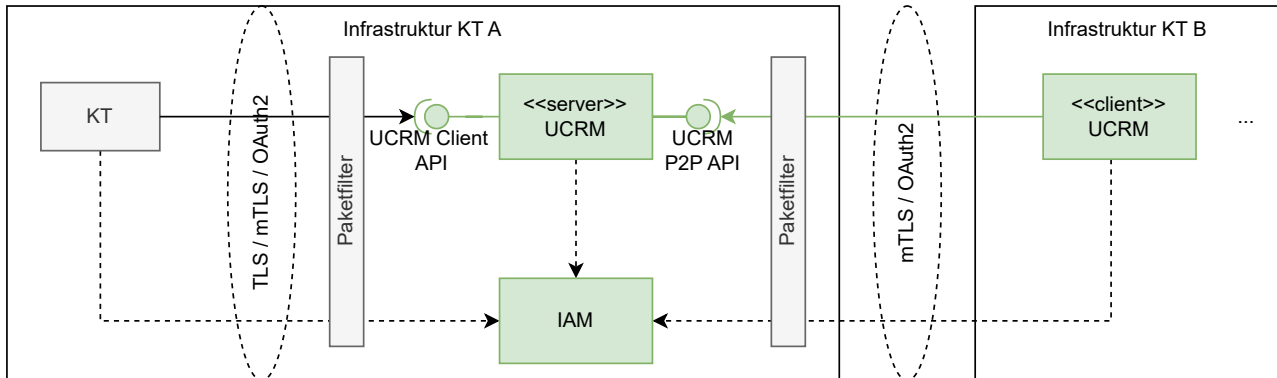
Einzelne Applikationen werden separat und unabhängig von der Spezifikation der Vermittlungsebene (Transportschichtspezifikation) versioniert. Siehe [UCRI2 Versionierung](#). Das erlaubt freie Weiterentwicklung jeder einzelnen App.

2.4. Sicherheitskonzept

Einzelne Sicherheitsaspekte werden im Folgenden im Hinblick auf eine P2P-Architektur betrachtet. Spezifika einer jeweiligen Broker-Architektur (nicht in Scope der vorliegenden Spezifikation) sind bei der Gestaltung der Inter-UCRM-Kommunikation zu berücksichtigen.

UCRI2 unterscheidet zwei Kommunikationsdomäne, siehe Abbildung:

1. Kommunikation zwischen einem Leitstellensystem (KT-System) und einem UCRI Leitstellenmodul (UCRM). Diese Kommunikation findet anhand der UCRM Client-API statt und erfolgt typischerweise innerhalb einer durch einen Leitstellenbetreiber kontrollierten Infrastruktur.
2. Kommunikation zwischen zwei UCRM. Die Inter-UCRM-Kommunikation in einer P2P-Architektur verwendet UCRM P2P-API. Bei einer Broker-Architektur (nicht abgebildet) findet Inter-UCRM-Kommunikation über das Broker-spezifische Protokoll statt. Die Inter-UCRM-Kommunikation kann über das öffentliche Internet stattfinden und braucht dementsprechend besondere Sicherheitsmaßnahmen.

P2P-Architektur

Die Infrastruktur eines KT sollte eine P-A-P-Struktur aufweisen, die aus Paketfilter als Trennung zu vertrauenswürdigen internen Systemen (Leitstelle), Application-Layer-Gateway (UCRM) und Paketfilter als Trennung zu nicht vertrauenswürdigen Netz (Internet) besteht. Vgl. [Netzarchitektur und -design - BSI](#).

UCRM API Implementierung verwendet in beiden Kommunikationsdomänen OAuth 2.0 mit Client Credentials Grant Type zur sicheren Authentifizierung und Autorisierung von Clients auf der Applikationsebene.

Trust Konzept

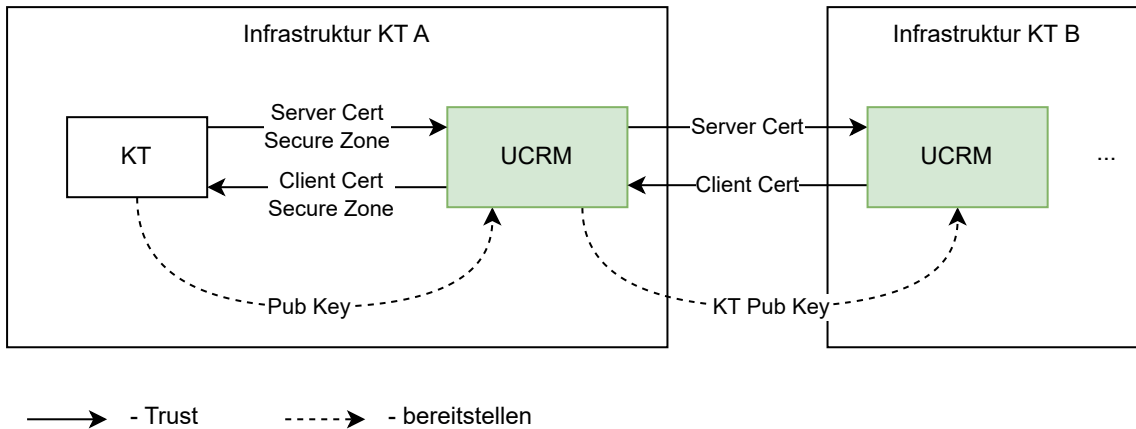
Bei einer P2P-Architektur wird mTLS als Sicherung der Inter-UCRM-Kommunikation zwischen zwei UCRM verwendet. Sowohl Client als auch Server bauen eine gegenseitige Vertrauensbeziehung über digitale Zertifikate auf. Dabei kann der Paketfilter auf der Seite des nicht vertrauenswürdigen Netzes die mTLS-Verbindung terminieren. Im Paketfilter kann eine Liste von zugelassenen Domainnamen implementiert (White Listing) und somit die Kommunikation auf eine geschlossene Gruppe der Teilnehmer beschränkt werden. Auch im Falle einer Broker-Architektur (nicht abgebildet) sollen entsprechende Mechanismen zur Herstellung der gegenseitigen Vertrauensbeziehung implementiert werden.

Die Kommunikation zwischen einem KT-System und dem UCRM-Modul kann je nach lokalen Sicherheitsanforderungen wahlweise per TLS oder mTLS erfolgen.

Die Vertrauensbeziehungen zwischen den Systemkomponenten bei der Client/Server-Kommunikation basieren dabei auf folgenden Mechanismen (siehe Abbildung):

1. Server-Authentifizierung. Die Vertrauensbeziehung zu UCRM als Server basiert auf dem Domainname (DNS) des Servers und wird durch Server-Zertifikat abgesichert. Der Domainname des Servers ist im Server-Zertifikat verankert.
2. Client-Authentifizierung. Die Vertrauensbeziehung zu KT oder UCRM in der Client-Rolle basiert auf der Object Identifier (OID) des Clients (siehe [Adressierungskonzept](#)) und wird durch Client-Zertifikat abgesichert. Die OID des Clients ist im Client-Zertifikat verankert.
3. Sowohl Client als auch Server haben eine Liste vertrauenswürdiger Zertifizierungsstellen (CAs), sogenannte Root-CAs, die als Vertrauensanker dienen. Die ausgestellten Zertifikate von Client und Server müssen von einer CA signiert sein, die jeweils in der vertrauenswürdigen Liste des Gegenübers enthalten ist.

P2P-Architektur



Anmerkung 1: In den konkreten Kundensituationen können unterschiedliche Mechanismen des Zertifikatsmanagements umgesetzt werden - basierend sowohl auf zentraler Authority als auch auf spezifischen Vereinbarungen zwischen einzelnen KT's.

Anmerkung 2: Als Alternative kann das Vertrauen zwischen KT-Systemen und dem UCRM in einer durch einen Leitstellenbetreiber kontrollierten Infrastruktur durch das Bilden von einer Sicherheitszone auf der Netzwerkebene hergestellt werden.

Um den sicheren Ursprung und unverfälschten Inhalt von Applikationsmeldungen zu garantieren, können die Meldungen durch Sender-KT signiert werden. Zur Signaturvalidierung erstellt der KT ein kryptographisches Schlüsselpaar und stellt sein Public Key dem lokalen UCRM-Modul über sicheren Kanal bereit. Bei der Inter-UCRM-Kommunikation wird das KT Public Key an die fremden UCRM mittels UCRM P2P-API übermittelt und steht anschließend den potentiellen Meldungsempfängern über die UCRM Client API zur Signaturvalidierung zur Verfügung. Das Signieren von Nachrichten ist im Kapitel [Kommunikationsprotokoll](#) detailliert beschrieben.

2.5. Versionierung

Die Versionierung für die Vermittlungsebene (UCRI2-Transportschicht) und die UCRI2-Apps erfolgen voneinander getrennt.

Diese Spezifikation beschreibt die Version 2.0.0 der UCRI2-Transportschicht.

2.5.1. Transportschicht-Versionierung

Die Transportschichtspezifikation umfasst dieses Dokument sowie die OpenAPI-Spezifikationen für die Client- und die P2P-Schnittstellen.

Die Versionsnummer für die Transportschichtspezifikation besteht aus drei numerischen Teilen:

GEN.MAJOR.MINOR

Für die drei Versionsbestandteile gelten folgende Festlegungen:

1. GEN : Für UCRI2 auf 2 festgelegt
2. MAJOR : Hauptversion der Transportschicht innerhalb der UCRI2-Versionierung. Eine Änderung dieser Version erfolgt, wenn Änderungen an den API-Endpunktdefinitionen erfolgen, die entweder bestehende Endpunkte bezüglich obligater Felder verändern oder neue obligate Felder hinzufügen.
3. MINOR : Unterversion der Transportschicht. Eine Änderung dieser Version erfolgt, wenn neue optionale Felder zu bestehenden Endpunkte hinzugefügt oder neue optionale Endpunkte hinzugefügt werden.

Somit sind Änderungen an der **MINOR** -Version stets kompatibel, so dass Systeme mit übereinstimmenden **GEN.MAJOR** -Versionen untereinander kommunizieren können, auch wenn sie unterschiedliche **MINOR** -Versionen aufweisen.

Transportschicht-Meldungen

Die Transportschicht verwendet eine UCRI2-App "Transportschicht-Meldungen" (`transport_layer_messages`). Diese App beschreibt Nachrichten, die auf der Transportschicht erstellt und von verbundenen UCRM sowie Clients konsumiert werden. Somit MUSS eine spezifische Version dieser App durch alle Clients sowie UCRM, welche die Version 2.0.0 der Transportschicht implementieren, ZWINGEND unterstützt werden.

Die hierfür zu unterstützende Version der Transportschicht-Meldungen-App ist 1.0.

2.5.2. App-Versionierung

Die App-Versionierung erfolgt für jede App individuell. Die Versionsnummer einer App besteht aus zwei numerischen Teilen:

MAJOR.MINOR

Für die zwei Versionsbestandteile gelten folgende Festlegungen:

1. **MAJOR** : Hauptversion der App. Eine Änderung dieser Version erfolgt, wenn neue obligate Nachrichten hinzugefügt werden oder in bestehenden Nachrichten obligate Felder hinzugefügt oder verändert werden.
2. **MINOR** : Unterversion der App. Eine Änderung dieser Version erfolgt, wenn neue optionale Nachrichten hinzugefügt werden oder in bestehenden Nachrichten optionale Felder hinzugefügt werden.

UCRI2-Protokoll beinhaltet ein KT-Register, in dem App-Versionen, die ein Kommunikationsteilnehmer unterstützt, publiziert werden. Obwohl die Änderungen an der **MINOR** -Version stets kompatibel sind, müssen auch **MINOR** -Varianten unter unterstützten Apps eines Teilnehmers explizit als angegeben werden.

3. Kommunikationsprotokoll

Aus der Sicht einer Leitstelle (KT-System) ist UCRI2 eine Client/Server Kommunikation gegen UCRI2-Systemkomponente Leitstellenmodul (UCRM). Ein KT-System (Client) baut eine Verbindung zum UCRM (Server) und konsumiert die sogenannte **UCRI2 Client-API**. Die UCRI2 Client-API ist somit die einzige Schnittstelle, gegen die eine Leitstellensoftware integriert werden muss, um mit anderen Leitstellen über UCRI2 kommunizieren zu können.

Je nach Organisation der Verbindung und des Datenflusses zwischen den einzelnen UCRM auf der Vermittlungsebene werden zwei Kommunikationstopologien unterschieden:

1. **Peer-to-Peer Topologie**: Dabei kommunizieren alle UCRM direkt miteinander. Jedes UCRM muss eine direkte Netzwerkverbindung zu jedem einzelnen Partner-UCRM aufbauen. Gleichzeitig bietet jedes UCRM eine Schnittstelle an - die sogenannte **UCRI2 P2P-API**, die von anderen Partner-UCRM konsumiert wird.
2. **Broker-Architektur**: Alle UCRM sind über eine individuelle Verbindung an eine zentrale Systemkomponente (Broker) angeschlossen. Um untereinander Daten auszutauschen brauchen einzelne UCRM keine direkte Netzwerkverbindungen untereinander, die Kommunikation läuft immer über den Broker. Die Kommunikationsschnittstelle zwischen UCRM und dem Broker ist spezifisch für jeweilige Broker-Technologie und wird deswegen in dieser Spezifikation nicht betrachtet.

Die Kommunikationstopologien können auch kombiniert werden.

Je nach Rolle in der Gesamtkommunikation ergibt sich damit folgende Relevanz der vorliegenden Spezifikation für die beteiligten Systemkomponenten:

1. Ein KT, der sich per UCRI2 mit anderen KT verbinden will, muss ausschliesslich die UCRI2 Client-API aus der Consumer-Perspektive umsetzen.

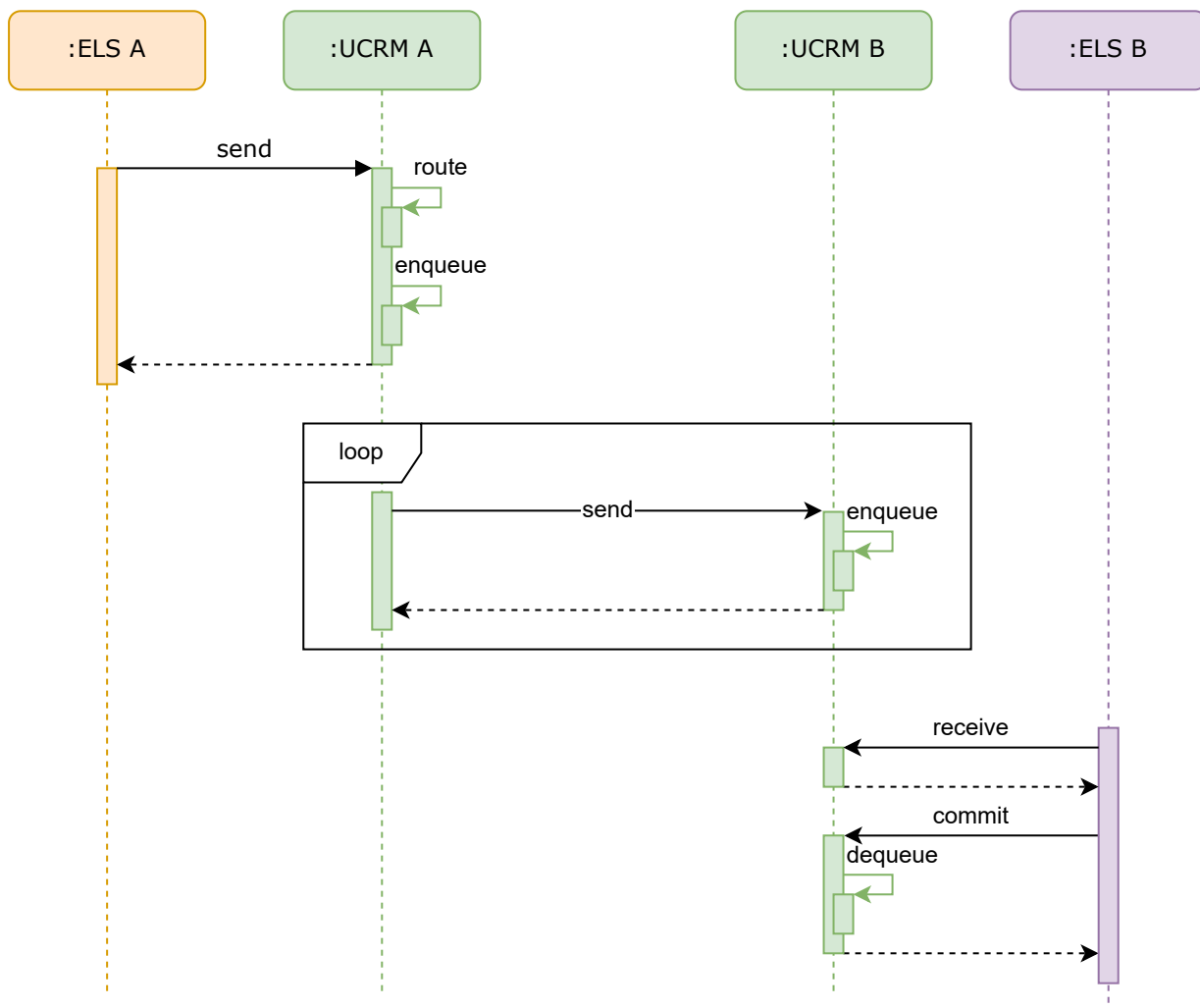
2. Ein UCRM muss die UCRI2 Client-API aus der Provider-Perspektive umsetzen. Außerdem muss ein UCRM, das an einem Kommunikationssystem mit P2P-Topologie beteiligt ist, die UCRI2 P2P-API umsetzen (Provider- und Consumer-Perspektiven). Im Falle einer Broker-Architektur muss CRM entsprechendes Broker-Kommunikationsprotokoll implementieren (nicht in Scope dieser Spezifikation).

Im Folgenden werden einzelne Aspekte des UCRI2-Kommunikationsprotokolls im Hinblick auf P2P-Topologie betrachtet. Hersteller von UCRM-Modulen für Broker-basierte Architekturen müssen die Inter-UCRM-Kommunikation unter Berücksichtigung von spezifischem Broker-Protokoll entsprechend anders umsetzen.

Die REST APIs Client-API und P2P-API werden [im Kapitel API](#) detailliert beschrieben.

3.1. Übermittlung von Nachrichten

Ein KT-System übergibt die ausgehenden Nachrichten an das UCRM-Modul m.H.v. Client-API-Endpunkt /send. Die Nachrichten werden dann durch lokales UCRM mittels P2P-API-Endpunkt /send gegen Empfänger-UCRM gepuscht. Für das Handling von Nichtverfügbarkeit der Partner-UCRMs wird in dem lokalen UCRM ein Ausgangspuffer implementiert.



3.2. Validierung von Nachrichten

Nachrichten werden in UCRM m.H.v. Nachrichtenschemata validiert. Die Validierung erfolgt synchron beim Senden. KT-Register liefert Auskunft über die durch KT unterstützten Applikationsschemata.

3.3. KT-Register

Zur Verwaltung der KTs dient ein KT-Register mit Systeminformationen (Systemname, Betreiber, technischer Support, etc.), Systemstatus, sowie Informationen über unterstützte UCRI2-Anwendungen eines Kommunikationsteilnehmers. Das KT-Register wird durch UCRM bereitgestellt und aktuell gehalten. Über Client-API steht der KT-Register den angeschlossenen KT-Systemen zur Verfügung.

Jedes UCRM baut einen lokalen Cache des KT-Registers durch regelmäßige Aufrufe des P2P-API-Endpunktes /registry an allen bekannten Partner-UCRM. Dabei liefert ein UCRM nur eigene direkt angeschlossene KTs.

Ein angeschlossenes KT-System fragt das KT-Register über den Client-API-Endpunkt /registry ab. Dabei liefert ein UCRM alle ihm bekannten (sowohl eigene, als auch über Inter-UCRM-Kommunikation ermittelten) KTs.

3.4. Datenintegrität

Die Kontrolle der Datenintegrität in Kommunikationssystemen ist von entscheidender Bedeutung, um sicherzustellen, dass die übertragenen Informationen vom richtigen Sender stammen und auf dem Übertragungsweg nicht manipuliert worden sind. Durch kryptographische Verfahren wie die **Signierung** wird die Authentizität und Unveränderbarkeit der Daten verifiziert, während die **Verschlüsselung** die Vertraulichkeit gewährleistet und den Inhalt vor unbefugtem Zugriff schützt.

Als Richtlinie für die Auswahl kryptographischer Verfahren für die Signierung und Verschlüsselung von Nachrichten dient die BSI Technische Richtlinie (https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Publikationen/TechnischeRichtlinien/TR02102/BSI-TR-02102.pdf?__blob=publicationFile&v=9).

Das asymmetrische kryptographische RSA-Verfahren RSASSA-PKCS1-v1_5 ([RFC 3447: Public-Key Cryptography Standards \(PKCS\) #1: RSA Cryptography Specifications Version 2.1](#)) wird sowohl zum Verschlüsseln als auch zum digitalen Signieren der Nachrichten verwendet.

Das Verfahren verwendet ein für jeden KT generiertes Schlüsselpaar, bestehend aus einem privaten Schlüssel und einem öffentlichen Schlüssel. Der private Schlüssel wird im Keystore des KT-Systems gespeichert.

Die öffentlichen Schlüssel werden über das KT-Register als Teil der verfügbaren KT-Daten in Form von JSON Web Key (JWK, [RFC 7517: JSON Web Key \(JWK\)](#)) ausgetauscht.

3.4.1. Signierung

Nachrichtensignaturen stellen sicher, dass eine Nachricht während der Übertragung nicht verändert wurde und dass sie von der angegebenen Quelle stammt. Hierzu sind zwei Schritte notwendig: Die Erzeugung eines eindeutigen Hash-Wertes für die Nachricht sowie die Signierung dieses Hashwertes durch das sendende System:

1. Für das Hashing wird die Nachricht zuerst gemäß JSON Canonicalization Scheme (JCS, [RFC 8785: JSON Canonicalization Scheme \(JCS\)](#)) in eine kanonische Form gebracht und diese kanonische Form dann mit SHA3-256 gehasht.
2. Für das Signieren und die Prüfung der Signatur wird das Verfahren nach dem IETF Standard JSON Web Signature (JWS, [RFC 7517: JSON Web Key \(JWK\)](#)) in der Variante Compact JWS in Kombination mit dem RSA-Verfahren RSASSA-PKCS1-v1_5 verwendet.

Diese Schritte werden im Folgenden genauer dargestellt.

Hashing der Nachrichten

Vor der Anwendung des JCS erstellt das signierende System eine Kopie der Nachricht, in der nur die folgenden Felder enthalten sind:

1. "source"
2. "destinations"
3. "payload"

Somit werden alle anderen Felder nicht beim Hashing berücksichtigt. Dies ist notwendig, da andere Felder freiwillig sind oder vom UCRM gesetzt werden, falls sie nicht vom Client gesetzt wurden. Als Hashing-Algorithmus kommt SHA3-256 zum Einsatz ([Use of the SHA3 One-way Hash Functions in the Cryptographic Message Syntax \(CMS\)](#)).

Signierung der Nachrichten

Als JWS-Header kommt ein Header mit Angabe des RSA-256-Algorithmus zum Einsatz:

```
{
  "typ": "UCRI_PLAIN",
  "alg": "RS256"
}
```

Der berechnete Hashwert der Nachricht wird als JWS-Payload verwendet. Dieser wird mit dem privaten Schlüssel des Nachrichtensenders signiert. Das Ergebnis (digitale Signatur) wird in Form einer JSON Web Signature (JWS, [RFC 7517: JSON Web Key \(JWK\)](#)) im Feld **signatur** übertragen:

```
BASE64(Header) || '.' || BASE64(Hash) || '.' || BASE64(Signatur)
```

Prüfung der Signaturen

Um die übermittelte JWS zu prüfen, MUSS das empfangende System wiederum

1. den Hashwert der empfangenen Nachricht berechnen
2. die übermittelte Signatur auf Gültigkeit prüfen
3. den signierten Hashwert (JWS-Payload) auf Gleichheit mit dem Hashwert der empfangenen Nachrichten prüfen

Die Ermittlung des Hashwertes erfolgt gemäß "Hashing der Nachrichten".

Die Signatur wird aus der empfangenen Nachricht extrahiert und gegen den im KT-Register hinterlegten öffentlichen Schlüssel des Senders validiert.

Das Signieren von Nachrichten ist optional. Das KT-Register beinhaltet Information, ob ein KT signierte oder nicht-signierte Meldungen sendet.

3.4.2. Verschlüsselung

Da jegliche Kommunikation zwischen KT und UCRM und zwischen UCRMs TLS verschlüsselt stattfindet, kann es auf eine E2E-Verschlüsselung des Nachrichteninhaltes verzichtet werden. Um die strengeren Sicherheitsanforderungen vor allem bei der Kommunikation über das Internet zu erfüllen, kann es später jedoch sinnvoll sein, den Nachrichteninhalt zusätzlich Ende-Zu-Ende, d.h. zwischen dem Sender-UCRM und dem Empfänger-UCRM oder sogar zwischen den KTs (in diesem Fall ohne Möglichkeit, die Meldungen durch UCRM validieren zu lassen), zu verschlüsseln.

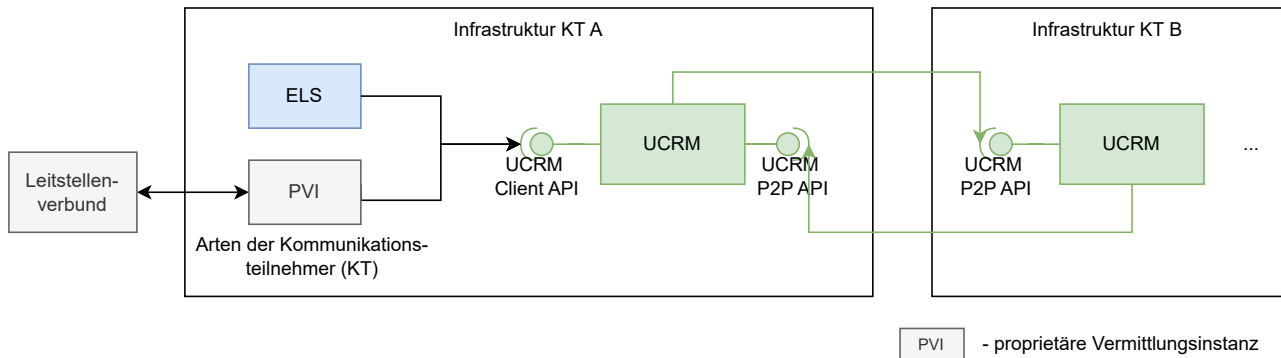
Für die Verschlüsselung wird dann das RSA-Verfahren RSASSA-PKCS1-v1_5 ([RFC 3447: Public-Key Cryptography Standards \(PKCS\) #1: RSA Cryptography Specifications Version 2.1](#)) verwendet.

Das konkrete Verschlüsselungsverfahren wird bei Bedarf später spezifiziert.

4. API

In diesem Kapitel wird die konkrete Umsetzung der UCRI2-APIs beschrieben. Gemäß dem Rollenkonzept werden zwei APIs unterschieden, siehe Abbildung:

1. Die **Client-API** wird vom **UCRM** angeboten und von **Clients** (Kommunikationsteilnehmern) konsumiert.
2. Die **P2P-API** wird von **UCRMs** angeboten, welche eine Verbindung mit anderen **UCRMs** auf Basis einer Peer-to-Peer-Topologie herstellen wollen.



Für beide APIs gelten folgende allgemeinen Festlegungen, welche in den jeweiligen Unterkapiteln mit API-spezifischen Festlegungen ergänzt werden:

1. Das grundlegende API-Design verwendet REST API Design Richtlinien erarbeitet bei [TM Forum](#).
2. Die konkrete Ausgestaltung der APIs inklusive der zu verwendenden Endpunkte und Datenformate wird in den zu der Spezifikation gehörenden OpenAPI-Spezifikationen in Version 3.1.0 (festgelegt in [OpenAPI Initiative](#)) festgelegt. Die OpenAPI-Spezifikationen sind strukturiert aufgebaut. Die übergeordneten API-Beschreibungen (`ucrm-client.yaml` sowie `ucrm-p2p.yaml`) verweisen jeweils auf die im Unterverzeichnis "schemas" vorhandenen `yaml` -Dateien mit den zu übertragenden Datenobjekten. Zusätzlich existieren noch inhaltsgleiche, gebündelte OpenAPI-Spezifikationen im JSON Format (`ucrm-client-bundled.json` sowie `ucrm-p2p-bundled.json`).
3. Beide APIs sind in folgende Funktionsbereiche aufgeteilt:
 - 2.1. Authentifizierung per OAuth 2.0 Client Credential Grant Flow.
 - 2.2. KT-Registry - Abfragen von Eigenschaften der registrierten Kommunikationsteilnehmer.
 - 2.3. Messaging - Versenden und Empfangen von Nachrichten.
 - 2.4. Info - Informationen über die Version und den Betreiber der Schnittstelle.
4. Die Datenübertragung erfolgt ausschliesslich per Transportverschlüsselung (TLS).
5. Die zur Übertragung von Fehlerzuständen genutzten Error-Objekte (vgl. `error.yaml`) werden im Unterkapitel [Fehlerbehandlung](#) für beide APIs gemeinsam dargestellt, da ein Großteil der genutzten Fehlercodes in beiden APIs vorkommt.
6. Die Generierung und Prüfung von Nachrichtensignaturen wird im Kapitel [Kommunikationsprotokoll](#) im Unterkapitel [Datenintegrität](#) beschrieben.
7. Falls ein Request nicht der OpenAPI-Spezifikation entspricht, MUSS dieser vom UCRM zurückgewiesen werden (vgl. Kapitel [Fehlerbehandlung](#)).
8. Ein UCRM MUSS die **Client-API** vollständig unterstützen.
9. Ein UCRM, welches die **P2P-API** implementiert, MUSS diese vollständig unterstützen.

4.1. Client-API

Die Client-API dient der Anbindung von KT (clients) an die Transportschicht (UCRM). Im Folgenden werden die API-Endpunkte vorgestellt:

4.1.1. /token - HTTP GET

Über den Token-Endpunkt werden JSON Web Token (JWT) abgerufen, die bei sämtlichen anderen Endpunkten zur Authorisierung genutzt werden. Hierzu werden eine Nutzerkennung sowie ein Geheimnis per HTTP-Header übergeben. Hierbei sei angemerkt, dass gemäß RFC 7519 ein **exp**-Claim als NumericDate dargestellt wird (also als die Sekunden seit 1970-01-01T00:00:00Z UTC), vgl. RFC 7519 Kapitel 2.

4.1.2. /info - HTTP GET

Über den Info-Endpunkt können allgemeine Informationen über das UCRM wie die genutzte UCRI2-Transportschicht-Version, UCRM-Hersteller, Produktversion und Betriebsstatus des UCRM abgerufen werden.

4.1.3. /registry - HTTP GET

Über den Registry-Endpunkt können KT Informationen über alle dem UCRM bekannten Kommunikationsteilnehmer (inklusive den UCRMs selbst) abrufen. Die Datenfelder verändern sich hierbei im Normalfall nur selten, eine Ausnahme bildet das **status**-Feld. Dieses gibt für einen KT an, ob dieser aktuell erreichbar ist. Der Wert dieses Feldes verändert sich je nach von der Vermittlungsebene bzw. den beteiligten UCRMs festgestellten Verbindungsstatus relativ dynamisch (vgl. [Verfügbarkeitsstatus unter API-Endpunkt /messaging/receive](#)). Da die Umgebung dynamisch ist, d.h. neue KT können dazukommen, existierende KT können abgebaut werden, sollen KT-Register-Abfragen regelmäßig durchgeführt werden (mindestens 1 Mal pro Stunde, maximal alle 5 Minuten).

4.1.4. /registry/{id} - HTTP GET

Über den Registry-ID-Endpunkt können KT Informationen über einen konkreten Kommunikationsteilnehmer abrufen, welcher die übergebene OID besitzt.

4.1.5. /messaging/send - HTTP POST

Über den Messaging-Send-Endpunkt können Nachrichten an andere KT versendet werden. Hierbei werden die Anwendungsnachrichten in eine sog. **Envelope** verpackt, welche Transportschicht-Metadaten enthält. Hierbei MUSS das UCRM folgende Prüfungen durchführen und im Fehlerfall einen passenden Error-Objekt zurückgeben:

1. Prüfung, ob der KT zum Senden der Nachricht berechtigt ist (Abgleich des source-Feldes mit dem übergebenen OAuth-Token).
2. Prüfung, ob der adressierte KT (destinations-Feld) bekannt ist.
3. Prüfung, ob die enthaltene Anwendungsnachricht (payload-Feld) eine bekannte Anwendung in einer bekannten Anwendungsversion referenziert und gemäß des Nachrichtenschemas gültig ist.
4. Prüfung, ob der adressierte KT (destinations-Feld) den Empfang der Anwendungsnachricht gemäß dessen KT-Registrierungs-Eintrages unterstützt.

Die Prüfung von Nachrichtensignaturen wird durch das UCRM für von clients gesendete Nachrichten NICHT durchgeführt, dies obliegt dem Empfänger der Nachricht.

Falls das UCRM eine unbekannte destination meldet, sollte der client über den /info-Endpunkt prüfen, ob das UCRM sich in einem normalen Betriebszustand befindet oder aktuell den initialen Abruf der KT-Registrierungsdaten von verbundenen UCRMs durchführt. In diesem Falle sollte abgewartet werden, bis das UCRM einen normalen Betriebszustand meldet.

Zustellungsquittierungen

Um die verlässliche Zustellung sicherzustellen, kann ein Client sowohl einen timeout-Wert für die Zustellung der Nachricht (**timeout**-Feld) setzen als auch eine explizite Empfangsbestätigung (**ack**-Feld) anfragen. In diesen Fällen überträgt das UCRM dem Client entweder eine negative Quittierung (falls ein Timeout aufgetreten ist oder bei der Weiterleitung der Nachricht innerhalb der Transportschicht ein Fehler aufgetreten ist) oder eine positive Quittierung (falls dies durch das **ack**-Feld angefragt wurde und der Empfänger den Empfang der Nachricht per Messaging-Commit-Endpunkt bestätigt hat). Hierzu kommt die **message_delivery_status**-Nachricht aus der **transport-layer-messages**-App zum Einsatz. Bezüglich dieser Statusbenachrichtigungen gelten folgende Festlegungen:

1. **message_delivery_status**-Nachrichten dürfen NUR durch UCRMs versendet werden, nicht durch clients. Dies muss durch das UCRM sichergestellt werden.
2. Ein UCRM sollte für eine versendete Nachricht MAXIMAL eine **message_delivery_status**-Nachricht an den KT zurückmelden.
3. Die Zuständigkeit für den Versand einer negativen Timeout-Quittierungs-Nachricht liegt IMMER bei dem UCRM, an welches der Sender angebunden ist. So wird sichergestellt, dass Timeout-Nachrichten auch dann generiert werden, wenn Übermittlungs-Probleme innerhalb der Transportschicht auftreten.

4.1.6. /messaging/receive - HTTP POST

Über den Messaging-Receive-Endpunkt können Nachrichten, die an den KT (bzw. eine Liste von OIDs im Feld **destinations**) adressiert sind, abgerufen werden. Diese Operation ist idempotent, kann also mehrmals mit dem gleichen Ergebnis wiederholt werden. Um den wiederholten Empfang einer Nachricht zu verhindern, MUSS der KT vor dem nächsten Abruf von Nachrichten die erfolgreich erhaltenen Nachrichten per Messaging-Commit-Endpunkt bestätigen.

Das UCRM MUSS prüfen, ob der Client zum Abruf der übergebenen OIDs berechtigt ist (durch Abgleich mit dem übergebenen OAuth-Token).

Das UCRM MUSS für jede empfangene Nachricht eine monoton steigende Sequenznummer im Feld **sequenceId** zurückgeben, die sich für eine individuelle Nachricht bei eventuellen mehrfachen Abrufen NICHT verändert.

Long-Polling

Da der Messaging-Receive-Endpunkt periodisch abgerufen werden muss, ist für einen schonenden Umgang mit Netzwerkressourcen und zur Sicherstellung möglichst geringer Zustellungsverzögerungen für diesen Endpunkt ein sog. [Long Polling](#) vorgesehen. Dieses MUSS vom UCRM unterstützt werden. Folgende Schleife ist bei Nachrichtenabfragen mittels Long Polling zu verwenden:

1. Nachrichten abfragen mit Angabe maximaler Nachrichtenzahl nMax (Feld **maxMessages**)
2. Falls Nachrichten gemeldet wurden, Nachrichten verarbeiten
3. Falls Nachrichten gemeldet wurden, Nachrichtenempfang bestätigen

Wichtig: bei Programmausfällen zwischen Schritten 2 und 3 kann es zum wiederholten Empfang von gleichen Nachrichten kommen. Die Client-Logik MUSS somit mit Nachrichtenduplikaten umgehen können. Dies kann durch Berücksichtigung der Eindeutigkeit der Nachrichten-ID (Feld **messageId**) geschehen.

Durch die Verwendung von Long Polling kann direkt ohne Pause von Schritt 3 zu Schritt 1 übergegangen werden. Für das Long Polling gelten folgende Festlegungen:

1. Die maximale Verzögerung (dMax) der Antwort beträgt 30 Sekunden.
2. Falls für mindestens eines der angefragten Ziele (Feld **destinations**) mindestens eine unbestätigte Nachricht vorliegt, wird die Anfrage sofort beantwortet.
3. Ansonsten wird die Beantwortung der Client-Anfrage solange verzögert, bis eine der folgenden Bedingungen vorliegt (die Bedingungen sind in der genannten Reihenfolge zu prüfen), dann wird die Anfrage ohne weitere Verzögerung beantwortet:
 1. Für mindestens eines der angefragten Ziele (Feld **destinations**) liegt mindestens eine unbestätigte Nachricht vor.
 2. Die maximale Verzögerung (dMax) der Antwort wurde erreicht.
4. Falls beim Long Polling Probleme auftreten, die sich nicht auf anderem Wege lösen lassen (z.B. durch die clientseitige Verlängerung von TCP-Timeouts), kann der Client mittels des Feldes **maxDelay** ein verkürztes dMax angeben, welches dann vom UCRM anstelle des in 1. genannten dMax zu verwenden ist.

Verfügbarkeitsstatus

Der periodische Abruf des Messaging-Receive-Endpunkts dient dem UCRM zusätzlich als Indikator über die aktuelle Erreichbarkeit des Client. Falls ein Client den Messaging-Send-Endpunkt für 60 Sekunden (Verfügbarkeits-Timeout) nicht abrufen, MUSS das UCRM den KT-Registrierungs-Verfügbarkeitsstatus (Feld **status**) auf "offline" setzen. Sobald wieder eine Anfrage eingeht, MUSS das UCRM den Verfügbarkeitsstatus auf "online" setzen. So können andere KT den aktuellen Verfügbarkeitsstatus via KT-Registrierung abrufen.

Durch die Wahl des Verfügbarkeitsstatus-Timeout (60 Sekunden) auf das Doppelte der maximalen Long-Polling-Verzögerung (30 Sekunden) ist sichergestellt, dass beide Funktionen miteinander harmonieren. Um aus Sicht der Vermittlungsebene als verfügbar zu erscheinen, MUSS ein Client also eine permanente Abfrage von Nachrichten durchführen (vgl. die im Kapitel "Long Polling" dargestellte Schleife).

4.1.7. /messaging/commit - HTTP POST

Über den Messaging-Commit-Endpunkt kann der erfolgreiche Empfang von Nachrichten für eine einzelne destination bestätigt werden. Hierzu wird eine Referenz-Sequenznummer angegeben (Feld **sequenceid**). Das UCRM entfernt dann alle Nachrichten mit Nachrichten-Sequenznummer $\leq$ Referenz-Sequenznummer aus der Liste unbestätigter Nachrichten und löst je nach Konfiguration des **ack**-Feldes die Übersendung einer Zustellbestätigung (**message-delivery-status**) aus. Falls ein Client Nachrichten für verschiedene OIDs abrufen, MUSS für jede Nachricht ein separater Aufruf des Messaging-Commit-Endpunktes durchgeführt werden.

Diese Operation ist idempotent, kann also mehrmals mit dem gleichen Ergebnis wiederholt werden.

Das UCRM MUSS prüfen, ob der Client zur Bestätigung von Nachrichten an die übergebenen OIDs (Feld **destination**) berechtigt ist (durch Abgleich mit dem übergebenen OAuth-Token).

4.2. Peer-to-Peer-API (P2P-API)

Die P2P-API dient der Vernetzung innerhalb der Transportschicht, also von **UCRMs** untereinander. Jedes beteiligtes UCRM bietet die API einerseits selbst an und verbindet sich andererseits mit den von entfernten UCRMs angebotenen APIs. Im Folgenden werden die API-Endpunkte vorgestellt:

4.2.1. /token - HTTP GET

Über den Token-Endpunkt werden JSON Web Token (JWT) abgerufen, die bei sämtlichen anderen Endpunkten zur Autorisierung genutzt werden. Hierzu werden eine Nutzerkennung sowie ein Geheimnis per HTTP-Header übergeben. Hierbei sei angemerkt, dass gemäß RFC 7519 ein **exp**-Claim als NumericDate dargestellt wird (also als die Sekunden seit 1970-01-01T00:00:00Z UTC), vgl. RFC 7519 Kapitel 2.

4.2.2. /info - HTTP GET

Über den Info-Endpunkt können allgemeine Informationen über das UCRM wie die genutzte UCRI2-Transportschicht-Version, UCRM-Hersteller, Produktversion und Betriebsstatus des UCRM abgerufen werden.

4.2.3. /registry - HTTP GET

Über den Registry-Endpunkt können entfernte UCRMs Informationen über alle direkt an das angerufene UCRM angebundenen Kommunikationsteilnehmer (inklusive den UCRMs selbst) abrufen. Hierbei MUSS das UCRM ausschliesslich die lokalen KT zurückgeben, nicht jedoch weitere über entfernte UCRMs angebundene KT.

Falls einzelne übertragenen KT-Registrierungsdatensätze fehlerhaft sind, MUSS das UCRM diese verwerfen.

Da die Umgebung dynamisch ist, d.h. neue KT können dazukommen, existierende KT können abgebaut werden, sollen KT-Register-Abfragen regelmäßig durchgeführt werden (mindestens 1 Mal pro Stunde, maximal alle 5 Minuten). Außerdem sollte ein UCRM beim Start die KT-Registrierungsdaten aller bekannten anderen UCRMs abrufen. Ein UCRM MUSS während dieser Phase den am Info-Endpunkt zurückgegebenen Status-Wert auf den Status 1 setzen (sowohl für die P2P- als auch für die Client-API).

Direkter Versand von Verfügbarkeits-Status-Nachrichten

Falls ein UCRM feststellt, dass sich der Verfügbarkeitsstatus eines Clients verändert hat, wird das KT-Registrierungs-**Status**-Feld aktualisiert (vgl. Unterkapitel "Client-API/Verfügbarkeitsstatus"). Diese Änderung wird dann entfernten UCRMs erst dann bekannt, sobald diese den nächsten periodischen Abruf der KT-Registrierung durchführen. Um die Statusveränderung direkt und ohne

Verzögerung an alle entfernten UCRMs zu kommunizieren, MUSS das UCRM daher zusätzlich nach Feststellung der Statusänderung eine **participant_availability_update**-Nachricht (aus der **transport_layer_messages**-App) an alle entfernten UCRMs senden.

Falls ein UCRM eine solche **participant_availability_update**-Nachricht erhält, MUSS es den KT-Registrierungs-Status des genannten KT auf den übermittelten Wert anpassen. Die Nachricht selbst darf NICHT an die angebundenen Clients weitergegeben werden.

4.2.4. /messaging/send - HTTP POST

Über den Messaging-Send-Endpunkt werden Nachrichten an entfernte UCRM weitergegeben. Hierbei gelten folgende Festlegungen:

1. Falls das entfernte UCRM die Nachrichtenannahme ablehnt und im **ack**-Feld nicht "none" angegeben war, MUSS das sendende UCRM dem KT, welcher die Nachricht ursprünglich gesendet hat, eine negative Zustellungs-Status-Nachricht übersenden, in deren **cause**-Feld der gemeldete Fehler mitübertragen wird.
2. Das empfangende UCRM MUSS prüfen, ob die Nachricht an einen der am UCRM angebundenen KT oder das UCRM selbst adressiert ist, falls dies nicht der Fall ist, MUSS die Nachricht abgelehnt werden.
3. Das empfangende UCRM MUSS prüfen, ob die enthaltene Anwendungsnachricht (payload-Feld) eine bekannte Anwendung in einer bekannten Anwendungsversion referenziert und gemäß des Nachrichtenschemas gültig ist.
4. Das empfangende UCRM MUSS prüfen, ob der adressierte KT (destinations-Feld) den Empfang der Anwendungsnachricht gemäß dessen KT-Registrierungs-Eintrages unterstützt.

4.3. Fehlerbehandlung

Falls als Antwort auf eine Anfrage ein Fehler zurückgegeben wird, enthält dieser IMMER einen numerischen, UCRI2-spezifischen Fehlercode. Dieser Fehlercode ermöglicht es, die Fehlerursache zusätzlich zur groben Fehlereinteilung per HTTP-Statuscode genauer festzustellen. Für jeden Fehlercode wird eine feste Fehlercode-ID vergeben, z.B. für den Fehlercode 460 die Fehlercode-ID REQUEST_INVALID_PER_CLIENT_TRANSPORT_SPEC (s.u.).

Andere als die unten genannten Fehlercodes dürfen NICHT zurückgegeben werden.

Falls dies nicht explizit angegeben ist, können alle Fehlercodes sowohl von der Client- als auch von der P2P-Schnittstelle zurückgegeben werden.

Zusätzlich vom Fehlercode wird IMMER eine menschenlesbare Fehlermeldung im reason-Feld mitübergeben. Die zusätzliche Übermittlung einer detaillierten Fehlerursache im message-Feld ist dagegen freiwillig.

Für jeden Fehlercode wird an dieser Stelle dargestellt, in welchen Situationen dieser Fehlercode zurückgegeben wird. Die Fehlercodes sind dabei nach den HTTP-Statuscodes geordnet, in deren Kontext sie auftreten. Fehlercodes dürfen NUR im Kontext des aufgeführten HTTP-Statuscodes übertragen werden. Eine Ausnahme bildet hierbei die Übertragung eines NAK-Zustellfehlers im Rahmen der message_delivery_status-Nachricht in der transport_layer_messages-App, wo der Fehlercode, der von einem entfernten UCRM über das P2P-Schnittstelle gemeldet wurde an den Client weitergeleitet wird.

4.3.1. Fehlercodes für Antworten mit HTTP-Statuscode 400

1.

460 (REQUEST_INVALID_PER_CLIENT_TRANSPORT_SPEC) : Dieser Fehlercode wird NUR von der Client-Schnittstelle zurückgegeben, falls der Request zwar valide JSON-Daten enthält, diese aber das Client-Transportschicht-Schema für diesen Request verletzen. Der Fehlercode kommt NICHT zum Einsatz, wenn ein enthaltener App-Payload das App-Schema verletzt (vgl. REQUEST_PAYLOAD_INVALID_PER_APP_SPEC).

2.

461 (REQUEST_PAYLOAD_UNKNOWN_APPID) : Dieser Fehlercode wird zurückgegeben, falls die angegebene AppID dem UCRM unbekannt ist.

3.

462 (REQUEST_PAYLOAD_UNKNOWN_APPVERSION) : Dieser Fehlercode wird zurückgegeben, falls dem UCRM zwar die AppId bekannt ist, aber die angegebene AppVersion unbekannt ist.

4.

463 (REQUEST_PAYLOAD_UNKNOWN_SCHEMAID) : Dieser Fehlercode wird zurückgegeben, falls dem UCRM zwar die AppId und AppVersion bekannt ist, aber die angegebene Schemald (also der Nachrichtentyp) unbekannt ist.

5.

464 (REQUEST_PAYLOAD_INVALID_PER_APP_SPEC) : Dieser Fehlercode wird zurückgegeben, falls der Request-Payload im data-Feld das zugehörige App-Schema verletzt.

6.

465 (REQUEST_PAYLOAD_INVALID_JSON) : Dieser Fehlercode wird zurückgegeben, falls es sich bei den übergebenen Daten nicht um gültiges JSON handelt.

7.

466 (REQUEST_PAYLOAD_UNSUPPORTED_APPID_OR_APPVERSION) : Dieser Fehlercode wird zurückgegeben, falls die appId und appVersion dem UCRM zwar bekannt, aber nicht in den supportedApps der destination als unterstützt aufgeführt sind.

8.

468 (REQUEST_PAYLOAD_UNSUPPORTED_MESSAGE) : Dieser Fehlercode wird zurückgegeben, falls die appId und appVersion dem UCRM bekannt und auch in den supportedApps der destination als unterstützt aufgeführt sind, die schemald (also der Nachrichtentyp) aber im entsprechenden supportedApps-Eintrag in der liste der unsupportedMessages aufgeführt ist.

9.

467 (REQUEST_PAYLOAD_FORBIDDEN_APPID) : Dieser Fehlercode wird NUR von der Client-Schnittstelle zurückgegeben, falls die gesendete Nachricht zur transport_layer_messages-App gehört (Nachrichten dieser App dürfen nur durch UCRMs versendet werden).

10.

470 (REQUEST_UNKNOWN_DESTINATION_ID) : Dieser Fehlercode wird zurückgegeben, wenn die angegebene destination dem UCRM nicht bekannt ist.

11.

478 (REQUEST_OID_FORBIDDEN) : Dieser Fehlercode wird zurückgegeben, falls der durch das übergebene OAuth-Token identifizierte Benutzer keinen Zugriff auf die angegebene OID hat. Bei der angegebene OID handelt es sich um die OID:

- 11.1. im sender-Feld (für den /messaging/sent-Endpunkt)
- 11.2. im destinations-Feld (für den /messaging/receive-Endpunkt)
- 11.3. im destination-Feld (für den /messaging/commit-Endpunkt)

12.

479 (REQUEST_WRONG_SIGNATURE) : Dieser Fehlercode wird NUR von der P2P-Schnittstelle zurückgegeben, falls es sich bei der gesendeten Nachricht um eine signierte Nachricht aus der transport_layer_messages-App handelt (und das sendende UCRM nicht per transmitsUnsignedMessages in seinem KT-Registrierungseintrag festgelegt hat, dass dieses seine Nachrichten nicht signiert). Auf der Client-Schnittstelle und generell für von Clients gesendete Nachrichten findet KEINE Signaturprüfung in der Transportschicht statt.

13.

480 (REQUEST_INVALID_PER_P2P_TRANSPORT_SPEC) : Dieser Fehlercode wird NUR von der P2P-Schnittstelle zurückgegeben, falls der Request zwar valide JSON-Daten enthält, diese aber das P2P-Transportschicht-Schema für diesen Request verletzen. Der Fehlercode kommt NICHT zum Einsatz, wenn ein enthaltener App-Payload das App-Schema verletzt (vgl. REQUEST_PAYLOAD_INVALID_PER_APP_SPEC).

4.3.2. Fehlercodes für Antworten mit HTTP-Statuscode 401

1. 475 (REQUEST_UNAUTHORIZED) : Dieser Fehlercode wird zurückgegeben, falls das übergebene OAuth-Token fehlt, ungültig oder expired ist. Außerdem wird der Fehlercode durch den /token-Endpunkt zurückgegeben, falls die übergebenen Zugangsdaten nicht korrekt sind.

4.3.3. Fehlercodes für Antworten mit HTTP-Statuscode 500

1. 491 (REQUEST_INTERNAL_ERROR) : Dieser Fehlercode wird zurückgegeben, falls ein interner Fehler aufgetreten ist.

5. Applikationen

Detaillierte Beschreibung der UCRI2-Applikationen befindet sich in dem Verzeichnis [docs/apps](#). Hierbei gibt das Dokument "UCRI2 App im Überblick" einen Überblick über die verschiedenen Apps und die PDF-Dateien in den Unterordnern beinhalten die detaillierte Dokumentation für verschiedenen Apps.